

 baidu/**Unlimited-OCR** 

like

1.56k

Follow

 BAIDU

2.92k

 Image-Text-to-Text  Transformers  Safetensors  multilingual unlimited-ocr

feature-extraction baidu vision-language ocr custom_code  Eval Results  arxiv:2606.23050

 License: mit

⋮

 Deploy ▾

→ Copy to bucket **NEW**

Use this model ▾

 **Model card**

Files

 xet

 Community 12

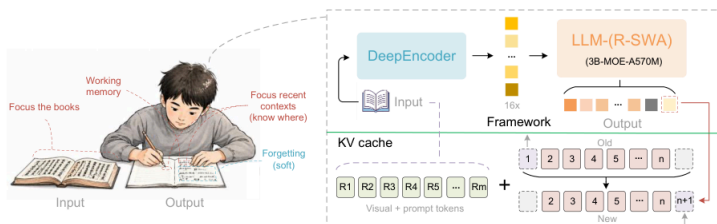


Unlimited OCR Works

 GitHub  Code  Hugging Face  Model

 arXiv **Unlimited OCR Works**  Twitter Baidu Inc.

Welcome the Era of One-shot Long-horizon Parsing.



Release

Downloads last

month

630,246



 **Safetensors** 

Model size 3B params

Tensor type BF16  Files info

 **Inference Providers** **NEW**

 Image-Text-to-Text

This model isn't deployed by any Inference Provider.

 28 Ask for provider support

 **Model tree for baidu/Unlimited-0...**

Finetunes 6 models

Quantizations  10 models

 **Spaces using baidu/Unlimited...** 19

 baidu/Unlimited-OCR

 akhaliq/Unlimited-OCR

- [2026/06/28] 🙌 Thanks to the [vLLM community](#) and [Tianyu Guo](#) for their support, our model now supports vLLM inference.
- [2026/06/24] 🙌 Thanks to [AK](#) for creating a demo for us. It is now available at [Hugging Face Spaces](#).
- [2026/06/23] 📄 Our paper is now available on [arXiv](#).
- [2026/06/23] 🙌 Thanks to the [ModelScope community](#) for their support. Our model is now available at [ModelScope](#).
- [2026/06/22] 🚀 We present [Unlimited-OCR](#), aiming to push [Deepseek-OCR](#) one step further.

Inference

Transformers

Inference using Huggingface transformers on NVIDIA GPUs. Requirements tested on python 3.12.3 + CUDA12.9 :

```
torch==2.10.0
torchvision==0.25.0
transformers==4.57.1
Pillow==12.1.1
matplotlib==3.10.8
einops==0.8.2
addict==2.4.0
easydict==1.13
pymupdf==1.27.2.2
psutil==7.2.2
```

 prithivMLmods/DeepSeek-OCR-2-Unl...

 Dlcastro/composa-unlimited-ocr-be...

 himipo/ocr-workbench-zeroGPU-Unli...

 youben44/unlimited-ocr

 profgabrielramos/Unlimited-OCR

 junwatu/ocr-man

+ 11 Spaces

 **Paper for** baidu/Unlimited-OCR

Unlimited OCR Works

 Paper • 2606.23050 • Publi... •  44

Evaluation results ⓘ

llamaindex/ParseBench 

Mean ⓘ  source	46.17 [*]
Text Content ⓘ  source	86.81 [*]
Text Formatting ⓘ  source	0.97 [*]
+3 more	

```

import os
import torch
from transformers import AutoModel, AutoTokenizer

model_name = 'baidu/Unlimited-OCR'

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(
    model_name,
    trust_remote_code=True,
    use_safetensors=True,
    torch_dtype=torch.bfloat16,
)
model = model.eval().cuda()

# — Single image supports two configs: gundam and base
# gundam: base_size=1024, image_size=640, crop_mode='center'
# base: base_size=1024, image_size=1024, crop_mode='center'
model.infer(
    tokenizer,
    prompt='<image>document parsing.',
    image_file='your_image.jpg',
    output_path='your/output/dir',
    base_size=1024, image_size=640, crop_mode='center',
    max_length=32768,
    no_repeat_ngram_size=35, ngram_window=128,
    save_results=True,
)

# — Multi page / PDF only uses base (image_size=1024)
model.infer_multi(
    tokenizer,
    prompt='<image>Multi page parsing.',
    image_files=['page1.png', 'page2.png', 'page3.png'],
    output_path='your/output/dir',
    image_size=1024,
    max_length=32768,
    no_repeat_ngram_size=35, ngram_window=1024,
    save_results=True,
)

```

```

# — PDF (convert pages to images, then multi-page PDF)
import tempfile, fitz  # PyMuPDF

def pdf_to_images(pdf_path, dpi=300):
    doc = fitz.open(pdf_path)
    tmp_dir = tempfile.mkdtemp(prefix='pdf_o')
    mat = fitz.Matrix(dpi / 72, dpi / 72)
    paths = []
    for i, page in enumerate(doc):
        out = os.path.join(tmp_dir, f'page_{i}.png')
        page.get_pixmap(matrix=mat).save(out)
        paths.append(out)
    doc.close()
    return paths

model.infer_multi(
    tokenizer,
    prompt='<image>Multi page parsing.',
    image_files=pdf_to_images('your_doc.pdf'),
    output_path='your/output/dir',
    image_size=1024,
    max_length=32768,
    no_repeat_ngram_size=35, ngram_window=10,
    save_results=True,
)

```

vLLM

Please refer to the official vLLM recipe for deployment details:

Recipe: <https://recipes.vllm.ai/baidu/Unlimited-OCR>

Docker Images

Use the following Docker images depending on your GPU platform:

Default (CUDA 13.0):

```
docker pull vllm/vllm-openai:unlimited-ocr
```

For Hopper GPUs (CUDA 12.9)

```
docker pull vllm/vllm-openai:unlimited-ocr-ci
```

SGLang

Set up the environment (uv-managed virtualenv).

Install the local SGLang wheel first, then pin

kernel==0.9.0 and install PyMuPDF for PDF-to-image conversion:

```
uv venv --python 3.12
```

```
source .venv/bin/activate
```

```
uv pip install wheel/sglang-0.0.0.dev11416+g'
```

```
uv pip install kernel==0.11.7
```

```
uv pip install pymupdf==1.27.2.2
```

Start the SGLang server:

```
python -m sglang.launch_server \
    --model baidu/Unlimited-OCR \
    --served-model-name Unlimited-OCR \
    --attention-backend fa3 \
    --page-size 1 \
    --mem-fraction-static 0.8 \
    --context-length 32768 \
    --enable-custom-logit-processor \
    --disable-overlap-schedule \
    --skip-server-warmup \
    --host 0.0.0.0 \
    --port 10000
```

Send streaming requests to the OpenAI-compatible API:

```
import base64
import json
import os
import tempfile

import fitz
import requests
from sglang.srt.sampling.custom_logit_processor import CustomLogitProcessor

server_url = "http://127.0.0.1:10000"

session = requests.Session()
session.trust_env = False

def pdf_to_images(pdf_path, dpi=300):
    doc = fitz.open(pdf_path)
    tmp_dir = tempfile.mkdtemp(prefix="pdf_images")
    mat = fitz.Matrix(dpi / 72, dpi / 72)
    image_paths = []
    for i, page in enumerate(doc):
        image_path = os.path.join(tmp_dir, f"page_{i}.jpg")
        page.get_pixmap(matrix=mat).save(image_path)
        image_paths.append(image_path)
    doc.close()
    return image_paths

def encode_image(image_path):
    ext = os.path.splitext(image_path)[1].lower()
    mime = "image/jpeg" if ext in (".jpg", ".jpeg") else "image/png"
    with open(image_path, "rb") as f:
        data = base64.b64encode(f.read()).decode()
    return {"type": "image_url", "image_url": f"data:{mime};base64,{data}"}

def build_content(prompt, image_paths):
    return [{"type": "text", "text": prompt}, {"type": "image_url", "image_url": f"data:image/jpeg;base64,{base64.b64encode(open(image_paths[0], 'rb').read()).decode()}"}, {"type": "text", "text": "Generate a short story about a cat."}]
```

```

def generate(prompt, image_paths, image_mode):
    payload = {
        "model": "Unlimited-OCR",
        "messages": [{"role": "user", "content": prompt}],
        "temperature": 0,
        "skip_special_tokens": False,
        "images_config": {"image_mode": image_mode, "image_paths": image_paths},
        "custom_logit_processor": DeepseekOcrLogitProcessor,
        "custom_params": {
            "ngram_size": 35,
            "window_size": ngram_window,
        },
        "stream": True,
    }
    response = session.post(
        f"{server_url}/v1/chat/completions",
        headers={"Content-Type": "application/json"},
        data=json.dumps(payload),
        timeout=1200,
        stream=True,
    )
    response.raise_for_status()

    chunks = []
    for line in response.iter_lines(chunk_size=1024):
        if not line or not line.startswith("data: "):
            continue
        data = line[len("data: "):]
        if data == "[DONE]":
            break
        event = json.loads(data)
        delta = event["choices"][0].get("delta")
        if delta:
            print(delta, end="", flush=True)
            chunks.append(delta)
    print()
    return "".join(chunks)

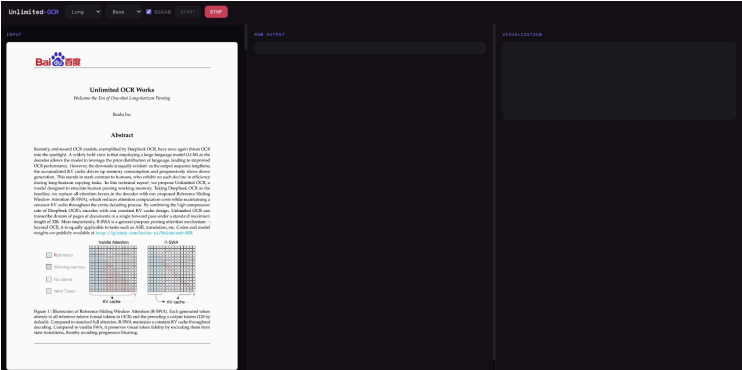
```

```
# Single image supports two configs: gundam ,
generate("document parsing.", ["your_image.jpg"])

# Multi image (base only)
generate("Multi page parsing.", ["page1.png"

# PDF (base only)
generate("Multi page parsing.", pdf_to_image:
```

Visualization



Acknowledgement

We would like to thank [Deepseek-OCR](#), [Deepseek-OCR-2](#), [PaddleOCR](#) for their valuable models and ideas.

Citation

```
@misc{yin2026unlimitedocrworks,
  title={Unlimited OCR Works},
  author={Youyang Yin and Huanhuan Liu and Yuhang He},
  year={2026},
  eprint={2606.23050},
  archivePrefix={arXiv},
  primaryClass={cs.CV},
  url={https://arxiv.org/abs/2606.23050}
```